Review of Scientific Instruments

RESEARCH ARTICLE | MAY 21 2026

Design and performance of AI agents interfacing with an atomic layer deposition tool

Angel Yanguas-Gil ; Jessica C. Jones ; Sungjoon Kim ; Chi Thang Nguyen ; Jeffrey W. Elam

Rev. Sci. Instrum. 97, 053903 (2026)
https://doi.org/10.1063/5.0318770

Review of Scientific Instruments ARTICLE pubs.aip.org/aip/rsi

Design and performance of AI agents interfacing with an atomic layer deposition tool

Cite as: Rev. Sci. Instrum. 97, 053903 (2026); doi: 10.1063/5.0318770
Submitted: 21 December 2025 • Accepted: 4 May 2026 •
Published Online: 21 May 2026

Angel Yanguas-Gil,[a] Jessica C. Jones, Sungjoon Kim, Chi Thang Nguyen, and Jeffrey W. Elam

AFFILIATIONS

Argonne National Laboratory, Applied Materials Division, Lemont, Illinois 60439, USA

[a]Author to whom correspondence should be addressed: ayg@anl.gov

ABSTRACT

In this work, we introduce the design of an atomic layer deposition (ALD) reactor augmented with an AI interface for autonomous materials synthesis. Our modular design encapsulates the particularities of the hardware behind a Python interface that communicates with the ALD control software via transmission control protocol. This interface is compatible with model context protocol interfaces used in agentic frameworks. We have integrated our tool with a simple AI agent that leverages a large language model to transform user-supplied queries into ALD processes that are then run in our reactor. Our approach uses a JavaScript object notation schema to encode ALD processes. Our experimental results show that the AI interface does not impose a significant overhead to our control software, at least within our fastest 10 ms scale. We also carried out a detailed evaluation of the agent performance using leading models in two classes of tasks: basic instruction and process discovery tasks, where the agent is presented with a target material and needs to identify the correct ALD process compatible with the reactor configuration. Despite the simplicity of our agent design, we observed that most of the advanced models excelled at the instruction tasks. However, only recent models, such as o1, o3, GPT-5, and Claude Opus 4, performed well in process discovery tasks. We also observed significant variability in the response for the hardest challenges. While the results obtained are promising, we identify areas where AI research could improve the performance of agents for ALD.

I. INTRODUCTION

In the recent years, there has been an increasing interest in AI agents capable of driving experiments as part of autonomous discovery platforms. However, the majority of experimental demonstrations, thus, far involve the use of customized experimental equipment.[1–3] Among thin film growth techniques, atomic layer deposition (ALD) is one prime candidate for autonomous systems interfacing with AI agents.[4,5] Due to the pulsed nature of ALD, ALD tools are already heavily automated, with control software actuating the valves required to realize the sequential dosing of precursors characteristic of ALD processes. Moreover, as a thin film technique, ALD is easy to integrate into larger cluster systems comprising synthesis and characterization modules. Finally, one of the main application domains of ALD is semiconductor processing, which historically has been heavily automated.

In the context of thin film growth, there have been a number of demonstrations of self-driving or autonomous materials synthesis, including techniques such as spin coating,[6] sputtering,[7,8] and pulsed laser deposition.[9] A significantly larger number of papers have focused on algorithm development. Most of the works in the literature with an experimental component use traditional machine learning approaches, such as Bayesian optimization.[6–10] Recently, however, agents built on top of large language models (LLMs) have shown significant promise to carry out more complex tasks in materials science and chemistry, such as process identification and orchestration of complex workflows,[3,11,12] that go beyond the capabilities of conventional algorithms. This is particularly true for so-called reasoning large language models, which have been shown to be capable of solving complex logic and math challenges.[13]

Despite their promise, there is very little evidence on the ability of these models to solve tasks relevant for thin film growth. While prior work has explored the performance of large language models when responding to queries about ALD,[14] and there are some preliminary results on the evaluation of agents' performance for the optimization of ALD processes,[15] how to integrate LLM-based AI agents with thin film growth tools and their performance on tasks

relevant for tool use or process discovery are questions that are still largely unexplored.

In this work, we address this gap in the literature, describing the design and integration of an AI agent with an ALD tool. We have carried out this integration in a pre-existing tool that needs to accommodate both humans and AI agents. This is representative of expected uses when existing manufacturing tools are augmented to integrate with AI. Starting from an existing ALD reactor,[16] we introduce our interface architecture with the AI interface. We also describe a basic agent architecture that leverages state of the art large language models to transform user-provided queries into processes that are autonomously run by the reactor. From an experimental perspective, we focus on benchmarking the computational overheads introduced by this AI layer and the response times of the AI agent. Finally, we benchmark the AI agent performance against different types of queries, comparing the performance of different large language models.

II. EXPERIMENTAL SETUP

A. ALD reactor

We have implemented our AI agent in the cross-flow ALD reactor shown in Fig. 1. From a hardware perspective, this reactor is almost identical to that described in Ref. 16, and we refer to that work for additional details on the hardware configuration. The reactor in Fig. 1(a) comprises two channels for delivering high pressure pyrophoric precursors, two channels for high pressure co-reactants, three channels for low vapor pressure precursors, and one channel for additional processing gases, such as ozone/oxygen or hydrogen.

Digital output operations required to actuate valves, analog inputs for temperature, pressure, and exhaust interlock measurements are controlled via a National Instruments cDAQ-9108 chassis with NI 9201, NI 9213, NI9435, and NI 9477 modules. This control design is similar to that of an *in situ* x-ray synchrotron ALD reactor that we have previously described,[17] and it represents the most significant change with respect to the experimental set up described in Ref. 16. Temperatures are set manually using Eurotherm and Omega temperature controllers, and flows are programmatically controlled via RS232 with an MKS 647C gas controller module. In Table I, we provide a list of main process variables and the current degree of autonomy. With the current hardware set up, all valves, mass flow controllers, and the exhaust valve pump down/vent cycles can be autonomously controlled. In particular, our tool allows us to create arbitrary sequences with custom-defined pulse and purge times at the core of complex ALD recipes.

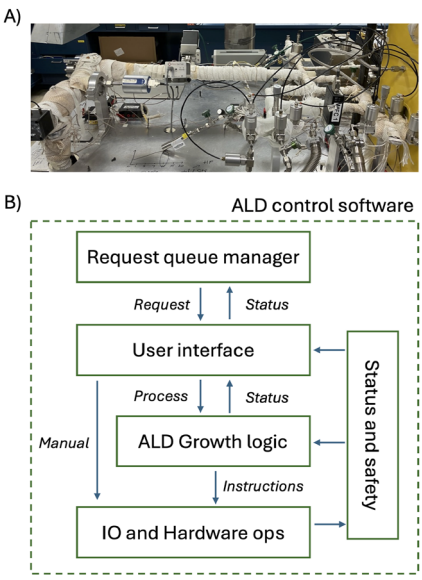

FIG. 1. (a) Picture of the ALD reactor. (b) Scheme of the control software, showing the main components and the logical relationships between them. This software is implemented in LabVIEW.

B. Control software

The reactor is controlled through a custom control software in LabVIEW. The most recent version of the software operates at three different levels of abstraction [Fig. 1(b)]: the lowest level takes care of all the I/O and hardware-specific operations, the intermediate level incorporates the logic of ALD growth, and the highest level of abstraction comprises the user interface, which is in charge of visualization, data logging, and communication with various *in situ* tools, such as quartz crystal microbalance, mass spectrometry, and spectroscopic ellipsometry. [We note that the *in situ* ellipsometer is visible in Fig. 1(a).] Information across all three levels is integrated via a subset of the code that evaluates the status of the reactor both for safety and to provide feedback to the growth logic. Communication between components at these three levels of abstraction takes place via function (subVI) calling, queues, and global variables.

In our design, ALD cycles and supercycles are the two key building blocks to design an ALD process. An ALD cycle is defined as an AB process comprising a "dose A–purge–dose B–purge" sequence common in simple ALD processes [e.g., Al_2O_3 ALD using (A) trimethyl aluminum and (B) water]. Sequences of AB cycles can then be arbitrarily combined to form a supercycle. This allows us to incorporate a wide range of specific cases, from the growth of doped and multicomponent materials to the use of inhibitors for area selective deposition and selective growth studies, or the use of multiple microdoses. Supercycles can be repeated as many times as needed to define an ALD growth.

For safety, the control software continuously monitors the reactor pressure, temperature, and pump purge flows to ensure that they are within safety limits. When conditions are deemed unsafe, growths are interrupted and no further experiments are allowed. The control software runs on an interface PC.

To allow for external communications, the ALD control software manages request and response queues. The request queue

TABLE I. Process variables and degree of autonomy in our ALD reactor.

Channel	Degree of autonomy
Valve actuation	Autonomous
Flows	Autonomous
Exhaust valve	Autonomous
Temperatures	Manual

manager consumes requests from a request queue that asks the control software for specific information, requests growths from the machine, or set specific conditions. Responses returned by the ALD reactor are pushed to the response queue and are meant to be consumed by external processes. These queues are accessible to any other LabVIEW programs running locally in the interface PC. The control software monitors the request queue with a wait and refresh period of 100 ms. This minimizes the impact on the reactor performance, leaving the fastest loops for IO operations to control valves and monitor the reactor conditions. Moreover, the request queue manager can be switched on/off by users to toggle between the AI and manual modes.

When a growth request is received in the request queue, the corresponding fields in the user interface are updated with the growth conditions and then the growth button is programmatically triggered via a value change (signal) property node in LabVIEW. This makes a growth request via the interface indistinguishable from a growth request entered by a human. From a safety standpoint, this means that the same interlocks and safety triggers used in our tools under manual operation are used in agentic requests.

C. AI interface layer

The modular architecture described in Sec. II B allows us to build an interface to enable agentic workflows in a way that minimizes the exposure of the critical components of the reactor to the external world. Since this reactor is meant to be shared between AI agents and human operators, one key design criterion was to ensure that AI integration did not negatively impact the reactor performance. To this end, we designed a modular interface layer with the architecture shown in Fig. 2. First, we created an ALD server component that implements a transmission control protocol (TCP) socket server to process external requests. This server is also built in LabVIEW, and it communicates with the ALD control software using the request and response queues described in Sec. II B. The ALD server runs in parallel to the ALD control software.

The ALD server is not allowed to communicate directly with external clients. Instead, we built a Python interface that mediates the interaction of the ALD reactor with outside requests. The Python interface is a Python module that, on the reactor side, implements a TCP socket client to communicate with the ALD server. This module provides a simple API to interact with the reactor, allowing us to take advantage of the wealth of machine learning and AI tools already available in Python. The same API can be used to interface with digital twins and functional models, for instance to refine the design of the agent and evaluate the performance of AI agents without the need to access an experimental reactor.

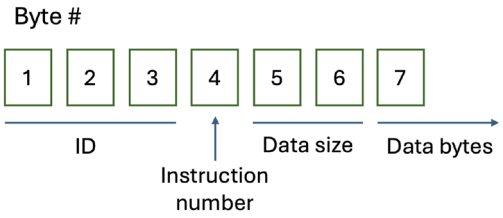

FIG. 3. Communication protocol of our ALD server. The ALD server is a TCP socket server that receives and sends messages using the structure shown in this figure.

The Python interface provides two ways for agents to communicate with the ALD tool: directly via the API and through a model context protocol (MCP) server, which provides a standardized way for AI agents to communicate with tools or resources.[18] While the API is more versatile and can be directly used to implement optimization algorithms, such as those described in our prior work,[10] MCP is an industry standard and it is directly geared toward interaction with generative AI models.

D. TCP communication protocol

The TCP communication protocol implements a minimal set of instructions required for an agent to work with an ALD reactor. The structure of the protocol is shown in Fig. 3. It comprises a six-byte header, where the first three bytes are an identifying code that is hardcoded in the ALD server, the fourth byte is the instruction number, and the last two bytes encode the length of the data in big-endian format. Data passed after the header is encoded in a JavaScript object notation (JSON) bytestring. We found that this ensures portability between Python and LabVIEW data structures. Failure to create a valid header terminates the connection. For this work, we focused on the minimal set of four instructions shown in Table II. This set is sufficient to implement AI agents that can reason about ALD processes based on the reactor configuration and feedback received.

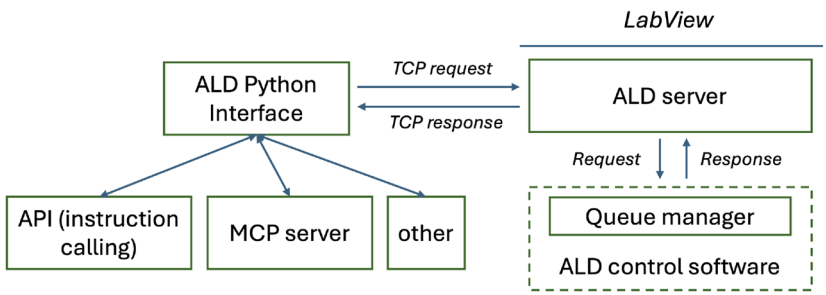

FIG. 2. High level diagram of our AI interface integrating our atomic layer deposition tool with the ALD control software. A queue manager in the control software processes external requests received from the ALD server, which, in turn, receives requests from our ALD Python interface. This interface is a standalone Python module that provides entry points to the ALD reactor via an API. It can be easily integrated as part of the backend of an MCP server or any custom AI algorithms.

TABLE II. Commands available to clients interfacing with our ALD server.

Instruction nos.	Name	Description	Return
0	Get status	Return the status of the reactor	Status data
1	Grow	Sends a recipe for grow	Acknowledgment
2	Get chemicals	Requests the current reactor configuration	Chemicals by channel
3	Get result	Request the result of the growth	Thickness and any additional parameters if growth is finished

E. Modes of operation

The architecture described in Secs. II B–II D allows us to create a remote access point that could integrate our tool into a larger autonomous materials synthesis lab. The communication workflow is structured as follows: a client request to the Python interface is translated into a TCP request (Table II) that is sent to the ALD server. The ALD server then pushes a request to the queues monitored by the queue manager implemented in the ALD control software. The result is then sent back to the ALD server via a response queue, and the ALD server responds to the Python client. The ALD server does not wait for an ALD growth to be completed before sending a response. Instead, it sends feedback on whether the request was processed correctly. It is up to the remote client to poll the Python interface to request updates about the growth status using the get status or get results commands. This ensures that updates can be sent while the growth is in progress, for instance, to monitor the output of *in situ* tools.

Another relevant use case for these agents is to simplify user interaction with the tool itself, where instead of manually operating the reactor interface, the human operator enters a text request to the reactor to carry out a specific growth. To allow for this use case, we have implemented a TCP client in the ALD control software that can send queries entered by the user directly to the AI agent. This uses the same protocol described in Sec. II D, except that the data sent is the query from the user and the response is just an acknowledgment that the AI server has received the request.

III. AGENT DESIGN

A. Architecture

A high level diagram of the architecture of the AI agent interfacing with our reactor is shown in Fig. 4. The agent has three components: the ALD interface to communicate with an ALD reactor, a large language model component that transforms text input into outputs, and a logic component that receives the query and orchestrates the logic and use of the AI model and experimental reactor. For some queries, additional background may be made available to the agent via a data module.

This approach lends itself to a highly modular design in Python, where an agent is constructed by passing two arguments—an *llm* object that handles communication with the LLM and the *ald_interface* object that implements the API to communicate with the reactor:

```
aldagent = ald_agent(llm, ald_interface)
```

A *query* is passed to the agent using a *run* method with any relevant prior data about the reactor (like logs of the growth) as an optional argument:

```
aldagent.run(query, history)
```

B. Logic component

In this work, we focus on a very simple example of an agent that executes a growth sequence based on an input query (e.g., "Grow ten cycles of Al_2O_3"). The agent is not allowed to carry out multiple growths to optimize the process and instead needs to accomplish this task with the information provided. Within this scenario, the structure of the logic component is very simple: after receiving the query, the logic component requests the configuration (a list with the precursor or chemical in each channel) from the ALD reactor. It then integrates the query, the reactor configuration, and any additional information provided as context for the agent via a data module into a prompt for the LLM, which provides a process matching the query. The logic component ensures that the response is in a format compatible with the API and sends a growth request to the ALD reactor, repeating interactions with the LLM as needed. For this specific work, the logic component maps the LLM response with a database of processes to select optimal dose and purge times (although see Ref. 15 for an example of an agent, still under evaluation, where the LLM component also provides dose and purge times for both the precursor and the coreactant).

C. LLM component

The role of the LLM component is to transform the complete request into a specific request to the reactor involving one or more independent ALD processes. In a prior work, we evaluated large language models for ALD using an open ended response benchmark.[14] Two of the conclusions obtained in that work were that, even though a model such as OpenAI's GPT4o was able to pass the benchmark, it struggled with quantitative responses and had a risk of hallucinating responses to queries about how to grow specific materials by ALD.

To mitigate this risk, we requested the model to provide the response in the form of a JSON schema containing the details

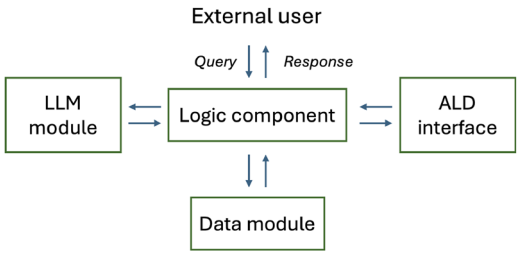

FIG. 4. Scheme of our agent: a logic component orchestrates access to an external LLM module, any relevant data made available to the data module, and the ALD Python interface described in Sec. II to answer external queries, which may come from a human user or from other agents.

TABLE III. List of AI models used in this work.

Model and version	Company	Use
GPT3.5	OpenAI	AI agent
GPT4o	OpenAI	AI agent
o1	OpenAI	AI agent
o3	OpenAI	AI agent
GPT5.0	OpenAI	AI agent
Claude Sonnet 4.0	Anthropic	AI agent
Claude Opus 4.0	Anthropic	AI agent
Claude Sonnet 4.5	Anthropic	AI agent
Gemini 2.5 Flash	Google	AI agent

required to carry out the synthesis process. For instance, for a conventional AB ALD process, we request the response to be in the form of a JSON dictionary with four fields: *precursor*, *coreactant*, *ncycles*, and *possible*. The values associated with these fields are the channel numbers for the precursor and the coreactant, the number of AB cycles that should be run, and a value of true/false depending on whether the request is compatible with the precursors installed on the reactor (i.e., the configuration). The model is asked to provide a response in the shape of a JSON list, where each element is a JSON dictionary for each ALD process. The examples of the expected JSON outputs are shown in Sec. IV.

Access to the various LLMs was done via API. For our implementation, we leveraged the internal generative AI interface available to researchers within Argonne National Laboratory. Private instances of multiple LLMs from OpenAI, Google, and Anthropic are available through Argo, offering a data-secure and experimentally controllable environment for research-grade LLM interactions. Table III summarizes the models evaluated in this work. No system prompt was provided for any of the models, so the LLM responses were intentionally not guided by additional directions beyond the information provided by the agent's prompt. Details on how to interface to the LLMs are encapsulated in the object passed to the agent so that the agent does not have to know the details about how the LLM is being accessed. Consequently, the same approach can be used when directly using access points to specific models, either online or local.

IV. RESULTS

A. Hardware performance

To test the performance of the AI agent described in Sec. III, we ran the query "Grow ten cycles of Al_2O_3" on an agent built on top of OpenAI's GPT4o model. The channels in the reactor included the strings "TMA" in Channel 0 and "water" in Channel 4. We repeated this experiment 50 times to gather statistics on the agent response, the success of the agentic workflow, and relevant execution times.

The results obtained showed that the agent successfully completed the whole workflow and correctly identified the correct configuration 100% of the time despite the probabilistic nature of output generation in LLMs. The LLM consistently returned the following valid JSON as a response:

```
[
{
"possible": 1,
"precursor": 0,
"coreactant": 4,
"ncycles": 10
}
]
```

This is remarkable given that at no point was the model explicitly shown in the prompt that "TMA" stands for trimethylaluminum or that trimethylaluminum/water is a viable precursor/coreactant combination for Al_2O_3 ALD. The model is, therefore, able to generate the correct output relying solely on GPT4o having been trained on the ALD literature. This intrinsic capability agrees with our prior evaluation results of GPT4o, which showed that this model had the ability to successfully generate answers in open-response benchmark focused on ALD.[14] The choice of JSON as an output seems to sidestep reproducibility issues, at least in the case of this specific prompt.

In Fig. 5, we show this sequence of events from an experimental standpoint: our reactor tracks the number of requests received via the ALD server. In Fig. 5(a), we show two consecutive requests received by the control software, separated by ~0.9 s. These correspond to the request for the reactor configuration and the request to initiate the growth. In Figs. 5(b) and 5(c), we show the reactor pressure during the initial growth showing transient features indicative of precursor vapor pulses and the status of the dose valves for the channels 0 and 4, which correspond to TMA and water. Only three of the ten cycles are shown in Figs. 5(b) and 5(c) for clarity. The delay between the request for growth being received and the growth stating is ~3.5 s. This is due to the logging procedure in our control software, which involves recording snapshots of all relevant screens at the beginning of each growth. This delay is, therefore, also present during user-initiated growths. Once the growth is started, there is no difference between an agent request and a user triggered growth.

In addition to evaluating the robustness of the agent's response, we also used this test to profile the agent's performance in terms of the response time, defined as the elapsed time between the moment the query is received by the agent and the time the agent receives the confirmation from the ALD server that the growth request is being processed. In Fig. 6(a), we show the distribution of response times for 50 independent queries. The average delay was 0.9 s with a standard deviation of 0.2 s.

The response time in Fig. 6(a) is the sum of times for three steps: two two-way communication instances between the Python client and the ALD server and one access from the AI agent to the remote LLM. In Figs. 6(b)–6(d), we show the distribution of these times for each of these steps for the same the 50 runs. As expected, the distributions of the two calls to the ALD reactor [Figs. 6(b) and 6(d)] have similar distributions. The average times are significantly smaller than the response time of GPT4o [Fig. 6(c)]. The mean delay times and standard deviations are shown in Table IV.

To quantify the impact of potential overheads introduced by the ALD server on the ALD control software, we compared the mean iteration time in one of the fastest loops in our control software with and without the ALD server. In our control software, pressure inputs are read inside a timed loop with a nominal iteration time of 10 ms.

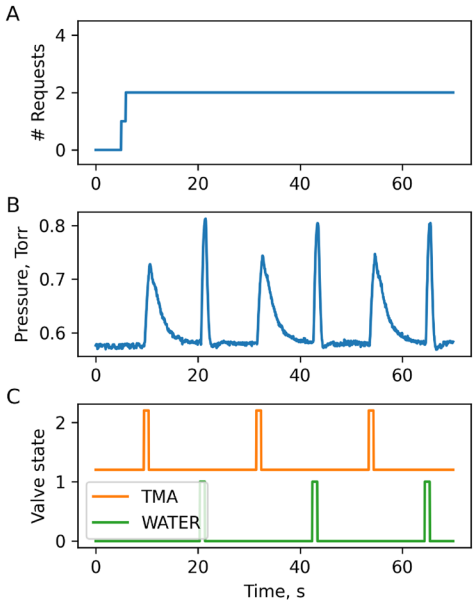

FIG. 5. Experimental data obtained during a query "Grow ten cycles of Al_2O_3" to the AI agent: (a) Accumulated number of requests received by the AI server. The first request is a status request to retrieve the channels and state of the reactor. The second request is the growth request. The delay between the two requests was 0.6 s; (b) total reaction pressure; (c) state of the TMA and H_2O dose valves.

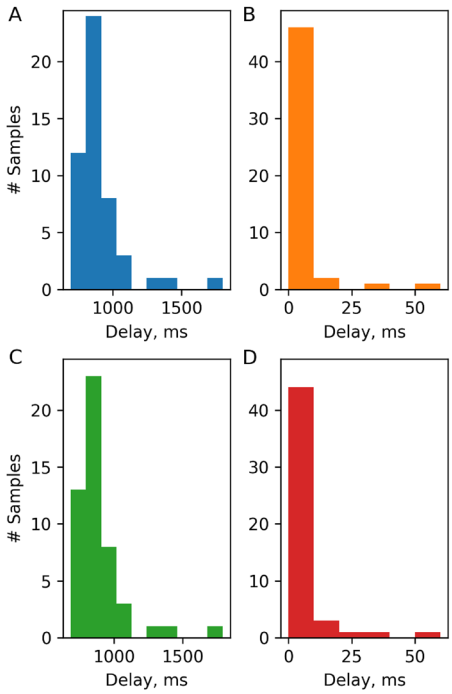

FIG. 6. Distribution of elapsed times between the agent receiving a request and the ALD tool acknowledging a growth request for the "Grow ten cycles of Al_2O_3." (a) Total response time, (b) response time for request to ALD server about reactor conditions; (c) response time of the LLM module; and (d) response time for the growth request.

TABLE IV. Response time statistics of our ALD tool for a simple request "Grow ten cycles of Al_2O_3." The response times shown are the average and standard deviation over 50 independent runs.

	Average	Standard dev.
First ALD server call	6 ms	8 ms
LLM call	0.9 s	0.2 s
Second ALD server call	11 ms	8 ms
Total	0.9 s	0.2 s

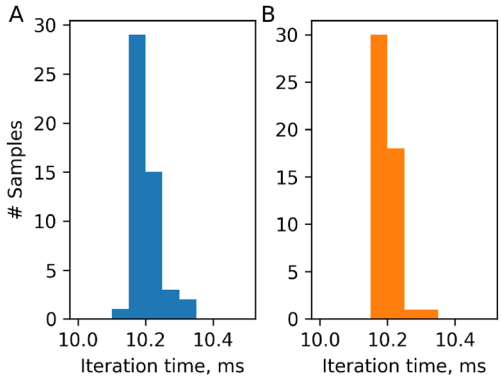

FIG. 7. Average iteration time for fast 10 ms loop when the ALD server is (a) inactive and (b) active. The results are consistent with a negligible overhead of the ALD server in the software controlling the ALD reactor.

We calculated the mean iteration time for this loop by counting the number of iterations over a 10 min interval. We then repeated this experiment 50 times to obtain a distribution of iteration times.

In Fig. 7, we show a comparison on the distribution of iteration times when the ALD server component is inactive [Fig. 7(a)] and active [Fig. 7(b)]. The results in Fig. 7 show that there is no measurable overhead imposed by the ALD server in the operation of the reactor.

Taken together, these results show that it is possible to augment the user interface of an ALD reactor for agentic work without significant penalties in performance nor significant delays beyond the natural response time with the remote LLM.

B. Agent performance

The tests run in Sec. IV A show promising results in terms of the agent's ability to identify a process from a user prompt. To better understand the capabilities of the AI Agent, we developed a benchmark to evaluate the performance of the AI agent in a more systematic way using a larger number of challenges.

These challenges comprise a query (e.g., "Grow 10 nm of ALD") and a reactor configuration to the LLM component of our AI agent in the same way as it does when interfacing with a real ALD reactor. Instead of executing a growth, we compare the model JSON response against a known correct answer. To quantify the quality of the response, we used a quantitative metric in a 0–1 scale, where 0 represents a failed response and 1 represents a correct response. If the sequence of cycles is correct, the response receives a score of 1.

TABLE V. Commands available to clients interfacing with our ALD server.

Challenge nos.	Query/configuration
1	*Query*: Please grow 200 cycles of TMA/water *Configuration*: TMA, water, DEZ, TDMAHf, Si_2H_6, WF_6, TTIP, $MgCp_2$
2	*Query*: Please grow 200 cycles of DEZ/water *Configuration*: TMA, water, DEZ, TDMAHf, Si_2H_6, WF_6, TTIP, $MgCp_2$
3	*Query*: Please grow 300 cycles of hafnia with TDMAHf and water *Configuration*: TMA, water, DEZ, TDMAHf, Si_2H_6, WF_6, TTIP, $MgCp_2$
4	*Query*: Please grow 300 cycles of WF6/Si_2H_6 *Configuration*: TMA, water, DEZ, TDMAHf, Si_2H_6, WF_6, TTIP, $MgCp_2$
5	*Query*: Please grow ten cycles of TMA/water followed by 20 cycles of WF_6/Si_2H_6 *Configuration*: TMA, water, DEZ, TDMAHf, Si_2H_6, WF_6, TTIP, $MgCp_2$
6	*Query*: Please grow ten cycles of TMA/water followed by ten cycles of TDMAHf/water and another ten cycles of TMA/water *Configuration*: TMA, water, DEZ, TDMAHf, Si_2H_6, WF_6, TTIP, $MgCp_2$
7	*Query*: Please grow 20 supercycles of 10xDEZ/H_2O + 1xTMA/H_2O *Configuration*: TMA, water, DEZ, TDMAHf, Si_2H_6, WF_6, TTIP, $MgCp_2$
8	*Query*: Deposit 50x (10xTMA/H_2O + 2xWF_6/Si_2H_6) *Configuration*: TMA, water, DEZ, TDMAHf, Si_2H_6, WF_6, TTIP, $MgCp_2$
9	*Query*: Functionalize the substrate with one cycle of Hacac followed by ten cycles of DMAI/water *Configuration*: DMAI, water, DEZ, Hacac, Si_2H_6, WF_6, TTIP, $MgCp_2$
10	*Query*: Dose one cycle of DMADMS followed by ten cycles of Ru(EtCp)$_2$/O_2 *Configuration*: TMA, water, Ru(EtCp)$_2$, O_2, DMADMS, Si_2H_6, WF_6

Otherwise, we grade the response based on the causes for this difference. If the response contains the wrong channel, the score is 0. If the channels are correct, but the number of cycles is wrong, the response is graded based on the relative error in the number of cycles. Finally, the agent's response is also graded a 0 if it is not in the correct JSON.

In this work, we have evaluated the performance of different AI agents implemented using the following large language models: GPT3.5, GPT4o, o1, o3, GPT5, Claude Sonnet 4, Claude Opus 4, Claude Sonnet 4.5, and Gemini 2.5 Flash. The results obtained are the average of five independent runs.

1. Requests involving process instructions

We first focused on a set of instruction challenges where we specifically ask the agent to carry out a series of growths using chemicals in the reactor's configuration. An example of this type of challenge would be a query such as "Please grow ten cycles of TMA/water." The queries for each of the challenges that we explored in this work are summarized in Table V. The full information, including the reactor configuration and expected response for each challenge, is provided in the supplementary material. This type of query is representative of the type of request expected from an agent that is part of an autonomous discovery lab. It is also representative of a text-based interface to control an ALD reactor. The queries shown in Table V are designed to cover a wide range of possible scenarios, from simple processes to complex sequences as well as the use of inhibitors for area selective deposition. We also tested the agent's ability to respond to requests involving more than one growth. For this challenge, we do not evaluate dose and purge times, as in our current agent design these are retrieved from a database of existing ALD processes (see Ref. 15 for an example of an agent based on reasoning language models used for the optimization of ALD processes).

In Fig. 8, we show the average scores for each of the challenges over five independent runs. Of the models tested, GPT4o, o1, o3, GPT5, Claude Sonnet 4, and Claude Opus 4 were all consistently able to correctly transform the queries into the right sequence of ALD cycles. Together with the performance results from Sec. IV A, these results indicate that the simple AI agent architecture presented in this work combined with the right model can act as a reliable

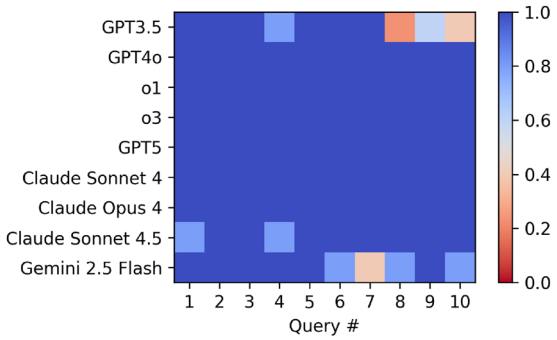

FIG. 8. Average performance over five independent runs of ALD agents based on different LLM models on instruction processing challenges. The corresponding queries are listed in Table V.

TABLE VI. Queries for process identification challenges with the corresponding reactor configuration. These gauge an agent's ability to identify and execute the right process based on a materials request.

Challenge nos.	Query
1	*Query*: Please grow 200 cycles of alumina *Configuration*: TMA, water, DEZ, TDMAHf, Si_2H_6, WF_6, TTIP, $MgCp_2$
2	*Query*: Please grow 200 cycles of ZnO *Configuration*: TMA, water, DEZ, TDMAHf, Si_2H_6, WF_6, TTIP, $MgCp_2$
3	*Query*: Please grow 300 cycles of hafnia *Configuration*: TMA, water, DEZ, TDMAHf, Si_2H_6, WF_6, TTIP, $MgCp_2$
4	*Query*: Please grow 300 cycles of tungsten *Configuration*: TMA, water, DEZ, TDMAHf, Si_2H_6, WF_6, TTIP, $MgCp_2$
5	*Query*: Please grow 250 cycles of TiO_2 *Configuration*: TMA, water, DEZ, TDMAHf, Si_2H_6, WF_6, TTIP, $MgCp_2$
6	*Query*: Please grow 350 cycles of MgO *Configuration*: TMA, water, DEZ, TDMAHf, Si_2H_6, WF_6, TTIP, $MgCp_2$
7	*Query*: Please grow 350 cycles of Er_2O_3 *Configuration*: TMA, water, DEZ, TDMAHf, Si_2H_6, WF_6, TTIP, $MgCp_2$
8	*Query*: Please grow 350 cycles of TiO_2 with a non-halogenated precursor *Configuration*: TMA, water, $TiCl_4$, TDMAHf, Si_2H_6, WF_6, TTIP, $MgCp_2$
9	*Query*: Please grow 200 cycles of Al_2O_3 *Configuration*: TMA, $TiCl_4$, DEZ, TDMAHf, Si_2H_6, WF_6, TTIP, $MgCp_2$
10	*Query*: Please grow 200 cycles of MoS_2 *Configuration*: TMA, water, DEZ, TDMAHf, Si_2H_6, MoF_6, TTIP, H_2S
11	*Query*: Please grow 250 cycles of zirconia *Configuration*: TMA, water, DEZ, TEMAZr, Si_2H_6, WF_6, TTIP, $MgCp_2$
12	*Query*: Please grow 200 cycles of alumina *Configuration*: DMAI, water, DEZ, TDMAHf, Si_2H_6, WF_6, TTIP, $MgCp_2$
13	*Query*: Please grow 200 cycles of erbium oxide *Configuration*: TMA, water, $Er(acac)_3$, TDMAHf, Si_2H_6, WF_6, TTIP, $MgCp_2$
14	*Query*: Please grow 300 cycles of hafnia with a carbon-free precursor *Configuration*: TMA, water, DEZ, TDMAHf, Si_2H_6, WF_6, $Hf(BH_4)_4$, $MgCp_2$
15	*Query*: Please grow 300 cycles of strontium oxide *Configuration*: TMA, water, DEZ, $Sr(CpiPr_3)_2$, Si_2H_6, WF_6, TTIP, $MgCp_2$
16	*Query*: Please grow 250 cycles of In_2O_3 with an alkyl precursor *Configuration*: TMA, water, $InMe_3$, $In(acac)_3$, Si_2H_6, WF_6, TTIP, $MgCp_2$
17	*Query*: Please grow 200 cycles of Osmium metal *Configuration*: TMA, water, DEZ, $OsCp_2$, Si_2H_6, WF_6, TTIP, $MgCp_2$
18	*Query*: Please deposit 350 cycles of Lithium sulfide *Configuration*: TMA, DEZ, water, Li(OtBu), SF_6, H_2S, TTIP, $MgCp_2$
19	*Query*: Please grow 20 supercycles of aluminum doped ZnO with a 9:1 DEZ/H2O:TMA/H_2O ratio *Configuration*: TMA, water, DEZ, TDMAHf, Si_2H_6, WF_6, TTIP, $MgCp_2$
20	*Query*: Please grow 200 cycles of Al:ZnO with 9:1 ZnO/Al_2O_3 ratio *Configuration*: TMA, water, DEZ, TDMAHf, Si_2H_6, WF_6, TTIP, $MgCp_2$
21	*Query*: Please grow 250 cycles of Mg doped ZrO_2 with a 1:9 cycle ratio starting with ZrO_2 *Configuration*: TMA, water, DEZ, TEMAZr, Si_2H_6, WF_6, TTIP, $MgCp_2$
22	*Query*: Please grow a nanolaminate comprising five bilayers of ten cycles of HfO_2 and ten cycles TiO_2 *Configuration*: TMA, water, DEZ, TDMAHf, Si_2H_6, WF_6, TTIP, $MgCp_2$
23	*Query*: Please grow a W/Al_2O_3 nanolaminate comprising 10x(20,2) cycles *Configuration*: TMA, water, DEZ, TDMAHf, Si_2H_6, WF_6, TTIP, $MgCp_2$
24	*Query*: Please grow 300 cycles of hafnia on top of 20 cycles of alumina *Configuration*: TMA, water, DEZ, TDMAHf, Si_2H_6, WF_6, TTIP, $MgCp_2$
25	*Query*: Please grow 200 cycles of MoS_2 and cap them with 20 cycles with Al_2O_3 *Configuration*: TMA, water, DEZ, TDMAHf, Si_2H_6, MoF_6, TTIP, H_2S
26	*Query*: Please functionalize the surface with 1 cycle of Hacac and then grow 20 cycles of Al_2O_3 using DMAI *Configuration*: DMAI, water, Hacac, TDMAHf, Si_2H_6, WF_6, TTIP, $MgCp_2$
27	*Query*: Please functionalize the surface with one cycle of Hacac and then grow 20 cycles of alumina *Configuration*: DMAI, water, Hacac, TDMAHf, Si_2H_6, WF_6, TTIP, $MgCp_2$
28	*Query*: Selectively deposit ten cycles of Ru using DMADMS as an inhibitor *Configuration*: TMA, water, $Ru(EtCp)_2$, O_2, DMADMS, Si_2H_6, WF_6
29	*Query*: Passivate the surface with hydrogen, functionalize it with DMATMS, and then grow 100 cycles of Ru *Configuration*: Tricarbonyl-(trimethylenemethane)-ruthenium, TMA, water, DMATMS, H_2, O_2, $TiCl_4$
30	*Query*: I want to grow a multilayer structure comprising a layer of 100 cycles of intrinsic MoS_2 and 100 cycles of Al:MoS_2 with a 9:1 MoS_2:Al ratio *Configuration*: TMA, water, DEZ, MoF_6, H_2S, Si_2H_6, WF_6

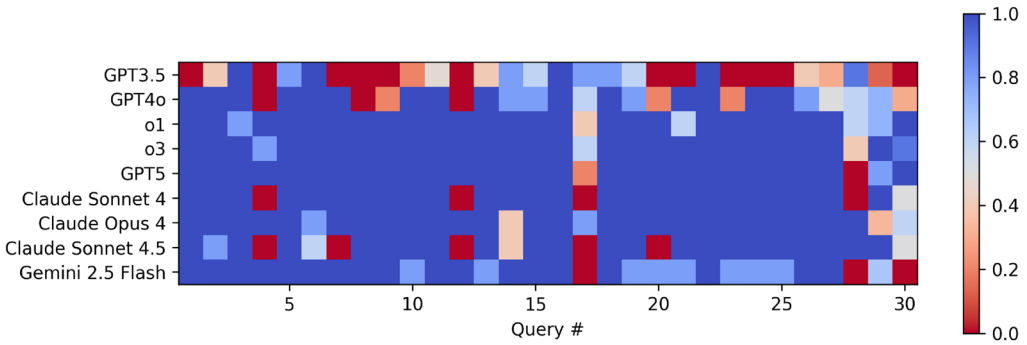

FIG. 9. Average performance over five independent runs of ALD agents based on different LLM models on process identification challenges. The corresponding queries are listed in Table VI.

interface that can consistently solve queries coming from either users or autonomous discovery platforms.

2. Requests involving process identification

To explore the limits of the agent, we also considered a set of process identification challenges evaluating the agents' ability to map queries involving specific materials with processes compatible with the reactor configuration. An example of this type of task would be the query "Grow 20 cycles of alumina" used in Sec. IV A. Here, we have considered 30 different challenges listed in Table VI. In all these cases, the AI agent is not provided additional information regarding the meaning of the precursors provided in the reactor configuration or information about any ALD process. It, therefore, relies on the underlying LLM's ability to generate the right process from the target material. The list of challenges and the correct responses is provided in the supplementary material.

In Fig. 9, we plot the scores for each of the questions for agents based on the same models used in Sec. IV B 1. The results are also the average performance across five independent runs. Of the models explored, o1, o3, GPT5, and Claude Opus 4 significantly outperformed the other models, with GPT3.5 being the least proficient at this subset of tasks. Lower average scores in the process identification challenges are primarily due to a higher variability on the quality of the response: due to the probabilistic nature of LLMs, lower performing models tend to provide incorrect answers, therefore lowering the average score.

To understand which challenges were the hardest, we calculated the average scores over all the agents. The results are shown in Fig. 10. A few challenges stand out: Challenges 17 and 4 involve two metal ALD processes, osmium and tungsten, respectively. One thing that both have in common is that they use non-conventional reducing agents as coreactant, oxygen in the case of osmium and disilane is the case of tungsten.[19,20] Challenge 12 involves a precursor, dimethyl aluminum isopropoxide,[21] that is significantly less common than trimethylaluminum. Challenge 28 also involves the growth of Ru metal using oxygen as a reducing agent and the use of an inhibitor, while challenge 30 involves a ternary sulfide process. These results suggest that, while agents built on commercially available, broadly trained large language models are surprisingly good at identifying the right process from growth requests, the performance tends to suffer for challenges involving processes that are less common.

To understand how the agent performance would improve when it has access to process information, we repeated the process identification benchmark except that now the model received a list of all ALD processes available in the reactor. The background information was passed for all the challenges (see supplementary material). In Fig. 11, we show the agents' performance with this additional information. We observe an improvement in the performance of the lowest scoring models, indicating that the additional information helps the agents generate the right processes.

Table VII summarizes the average scores for the different agents for the three studies carried out in this work: requests involving process instructions, requests involving process identification, and requests involving process identification with additional data on ALD processes. The addition of the background information significantly increases the average score of the lowest performing models for the process identification task. The overall scores, however, are still lower than in the process instruction case.

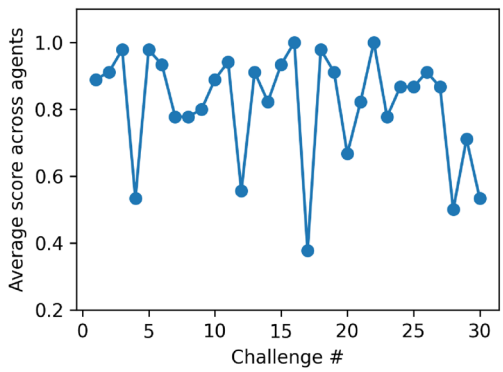

FIG. 10. Average score in each of the process identification challenges (Table VI) across the different LLMs used to test to build our AI agent.

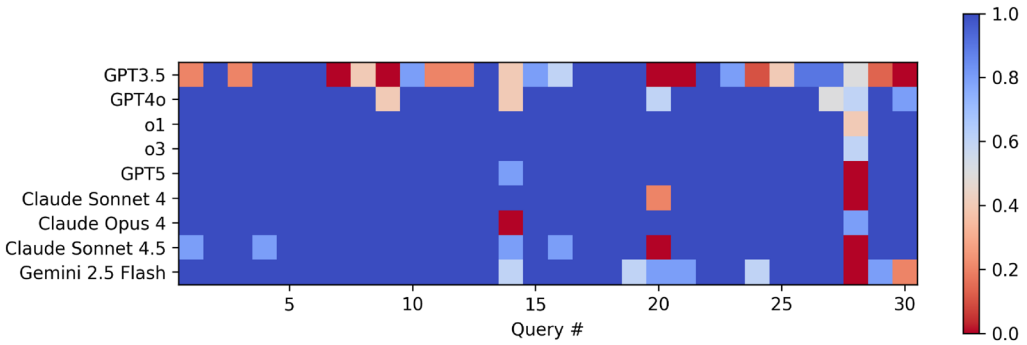

FIG. 11. Average performance over five independent runs of ALD agents based on different LLM models on process identification challenges when additional process information is provided to the agent. The corresponding queries are listed in Table VI.

TABLE VII. Average scores for the different agents as a function of the underlying large language model used for instruction challenges (Instruction), process identification challenges (Identification), and process identification challenges with additional background information (Identification + BG).

Model	Instruction	Identification	Identification + BG
GPT3.5	0.80(0.09)	0.39(0.04)	0.55(0.06)
GPT4o	1.0(0)	0.72(0.04)	0.91(0.03)
o1	1.0(0)	0.94(0.04)	0.98(0.02)
o3	1.0(0)	0.96(0.03)	0.99(0.02)
GPT5	1.0(0)	0.93(0.01)	0.96(0.01)
Claude Sonnet 4	1.0(0)	0.85(0.02)	0.94(0.01)
Claude Opus 4	1.0(0)	0.93(0.02)	0.96(0.01)
Claude Sonnet 4.5	0.96(0.05)	0.78(0.02)	0.91(0.05)
Gemini 2.5 Flash	0.88(0.07)	0.84(0.06)	0.88(0.04)
Average	0.96(0.07)	0.82(0.17)	0.90(0.13)

V. DISCUSSION

The results presented in Sec. IV show that it is possible to augment ALD tools so that they can effectively interface with AI agents with minimal overhead. The specific agent architecture explored in this work shows excellent performance for queries involving process-related instructions, with robust performance observed across a broad range of commercially available large language models. These results are promising both for the development of novel prompt-based user interfaces and the integration of these AI agents as part of larger autonomous materials discovery platforms where these reactor-specific agents would be a component of multiagent systems.

In contrast, the performance of AI agents in process identification tasks, where the agent needs to infer the ALD process from the reactor configuration, shows a stronger dependence with the type of model. In general, reasoning models, such as o1 or o3, tend to perform better than models such as GPT4o. While the performance of the best performing models is promising, scoring over 0.9 in a 0 to 1 scale, we observe that models overall tend to struggle with challenges involving processes that are either less commonly used or have unexpected chemistries (e.g., using oxygen as a reducing agent). This decrease in performance is not surprising given the limitations of the current agents: (1) they are based on commercial models and, therefore, rely on training data of these generic models; (2) they are not augmented using strategies such as retrieval augmented generation (RAG) that have been shown to improve the performance on scientific tasks. When we provide additional information on possible ALD processes, we observe an increase in performance, particularly among the lower performing models. We expect that the development of new models that can support agents for materials synthesis would further improve the agent performance in the more complex queries.

Our approach relies on encapsulating the complexities of hardware control, which in our case is based in LabVIEW, behind a simple Python interface that can be integrated with existing AI workflows. The use of JSON as a way of output mitigates the non-deterministic nature of the large language models. We believe that this approach can improve the reproducibility of the agent's responses. It also brings materials synthesis and process design closer to structured data generation tasks that LLMs are well suited for.[22] As shown in Sec. IV B, structured outputs are also useful from an evaluation standpoint, helping bridge existing gaps on the evaluation of LLMs for scientific applications.[23]

In this work, we have focused on a very simple agent engaged in zero-shot tasks, where the agent is not allowed to interact with the tool to test hypothesis before producing the desired outcome. We have not tried to mitigate errors from the LLMs by integrating other agents specialized in evaluating and rejecting incorrect responses. Consequently, there are significant opportunities in AI research to develop more performant, robust agents. One key area of improvement is in the agent design itself: more complex workflows can break down requests into subtasks, for instance to first identify the desirable processes, then to map them onto the reactor configuration, and finally to use a specialized agent to generate the JSON from the request. This strategy is commonly employed in the literature.[12] For instance, currently, we are exploring agents that can leverage the *in situ* tools present in our ALD reactor to provide real time feedback to the AI agent, and this will be the focus of a future work. A second area of improvement is using more sophisticated models that either use strategies such as retrieval augmented generation approaches to access the knowledge required to solve specific challenges,[24] or have been fine-tuned on ALD synthesis and process development data.[25]

Finally, we would like to emphasize the impact of the probabilistic nature of the LLM response: the results presented in this work for both instruction and process discovery challenges are the average of five independent runs. When we look at the individual responses, we observe that in most of the cases, lower scores reflect a lower probability of the model identifying the right process (as shown in Sec. IV B, the rubric used in this work to evaluate the agent's response scores a zero if the wrong process is used). While in this work we have not introduced any strategies to reduce such variability, understanding how *consistently good* agents based on LLMs are is critical before they are deployed in autonomous and self-driving labs.

VI. CONCLUSIONS

In this work, we have introduced an AI interface that can augment an existing atomic layer deposition tool for autonomous materials synthesis based on AI agents without significant overheads. Simple AI agents based on commercial large language models show excellent performance in instruction tasks and promising results for process identification tasks. While the performance of state of the art LLMs in process identification tasks is not perfect, there is still substantial margin for improvement in terms of agent design and AI research. To facilitate these efforts, we have created aldenv, a digital twin of our reactor that implements the architecture described in Secs. II and III and that includes the challenges described in this work. The benchmarks and agents are available at https://github.com/aldsim/aldenv.

SUPPLEMENTARY MATERIAL

The supplementary material includes the JSON structure for the instruction and process identification challenges, instruction prompts for running the challenges, and additional background information supplied to the model as hints for the process identification challenges.

ACKNOWLEDGMENTS

This research is based upon work supported by Laboratory Directed Research and Development (LDRD) funding from Argonne National Laboratory, provided by the Director, Office of Science, of the U.S. Department of Energy under Contract No. DE-AC02-06CH11357.

AUTHOR DECLARATIONS

Conflict of Interest

The authors have no conflicts to disclose.

Author Contributions

Angel Yanguas-Gil: Conceptualization (lead); Data curation (lead); Funding acquisition (lead); Investigation (equal); Methodology (lead); Software (lead); Supervision (equal); Validation (lead); Writing – original draft (lead); Writing – review & editing (lead). **Jessica C. Jones**: Data curation (supporting); Investigation (supporting); Writing – original draft (supporting); Writing – review & editing (supporting). **Sungjoon Kim**: Data curation (supporting); Investigation (supporting); Writing – original draft (supporting); Writing – review & editing (supporting). **Chi Thang Nguyen**: Data curation (supporting); Investigation (supporting); Writing – original draft (supporting); Writing – review & editing (supporting). **Jeffrey W. Elam**: Data curation (supporting); Resources (supporting); Supervision (supporting); Writing – original draft (supporting); Writing – review & editing (supporting).

DATA AVAILABILITY

The data that support the findings of this study are available within the article and its supplementary material.

REFERENCES

[1] F. Häse, L. M. Roch, and A. Aspuru-Guzik, Trends Chem. **1**, 282 (2019).
[2] N. J. Szymanski, B. Rendy, Y. Fei, R. E. Kumar, T. He, D. Milsted, M. J. McDermott, M. Gallant, E. D. Cubuk, A. Merchant, H. Kim, A. Jain, C. J. Bartel, K. Persson, Y. Zeng, and G. Ceder, Nature **624**, 86 (2023).
[3] M. C. Ramos, C. J. Collison, and A. D. White, Chem. Sci. **16**, 2514 (2025).
[4] E. Kessels, A. Devi, J.-S. Park, M. Ritala, A. Yanguas-Gil, and C. Wiemer, Nat. Rev. Methods Primers **5**, 66 (2025).
[5] P. Navabi, R. Ampadi Ramachandran, H. Bhatia, M. Jaberi-Douraki, U. Diwekar, C. Sukotjo, and C. G. Takoudis, J. Vac. Sci. Technol. A **43**, 060801 (2025).
[6] B. P. MacLeod, F. G. L. Parlane, T. D. Morrissey, F. Häse, L. M. Roch, K. E. Dettelbach, R. Moreira, L. P. E. Yunker, M. B. Rooney, J. R. Deeth, V. Lai, G. J. Ng, H. Situ, R. H. Zhang, M. S. Elliott, T. H. Haley, D. J. Dvorak, A. Aspuru-Guzik, J. E. Hein, and C. P. Berlinguette, Sci. Adv. **6**, eaaz8867 (2020).
[7] R. Toyama, R. Tamura, S. Matsuda, Y. Iwasaki, and Y. Sakuraba, npj Comput. Mater. **11**, 329 (2025).
[8] S. Jarl, J. Sjölund, R. J. W. Frost, A. Holst, and J. J. S. Scragg, Mater. Des. **260**, 115087 (2025).
[9] S. B. Harris, R. Y. López Fajardo, A. A. Puretzky, K. Xiao, F. Bao, and R. K. Vasudevan, Nano Lett. **25**, 2444 (2025).
[10] N. H. Paulson, A. Yanguas-Gil, O. Y. Abuomar, and J. W. Elam, ACS Appl. Mater. Interfaces **13**, 17022 (2021).
[11] S. Yu, N. Ran, and J. Liu, Artif. Intell. Chem. **2**, 100076 (2024).
[12] A. Ghafarollahi and M. J. Buehler, "Autonomous inorganic materials discovery via multi-agent physics-aware scientific reasoning," arXiv:2508.02956 [cond-mat.mtrl-sci] (2025).
[13] M. Balunovic, J. Dekoninck, I. Petrov, N. Jovanovic, and M. Vechev, "MathArena: Evaluating LLMs on uncontaminated math competitions," arXiv:2505.23281 [cs.AI] (2025).
[14] A. Yanguas-Gil, M. T. Dearing, J. W. Elam, J. C. Jones, S. Kim, A. Mohammad, C. Thang Nguyen, and B. Sengupta, J. Vac. Sci. Technol. A **43**, 032406 (2025).
[15] A. Yanguas-Gil, "Performance of AI agents based on reasoning language models on ALD process optimization tasks," J. Vac. Sci. Technol. A (in press) (2026).
[16] J. W. Elam, M. D. Groner, and S. M. George, Rev. Sci. Instrum. **73**, 2981 (2002).
[17] J. A. Klug, M. S. Weimer, J. D. Emery, A. Yanguas-Gil, S. Seifert, C. M. Schlepütz, A. B. F. Martinson, J. W. Elam, A. S. Hock, and T. Proslier, Rev. Sci. Instrum. **86**, 113901 (2015).
[18] H. Pan, R. Chard, R. Mello, C. Grams, T. He, A. Brace, O. P. Skelly, W. Engler, H. Holbrook, S. Y. Oh, M. Gonthier, M. Papka, B. Blaiszik, K. Chard, and I. Foster, "Experiences with model context protocol servers for science and high performance computing," arXiv:2508.18489 [cs.DC] (2025).
[19] J. Hämäläinen, T. Sajavaara, E. Puukilainen, M. Ritala, and M. Leskelä, Chem. Mater. **24**, 55 (2012).
[20] J. W. Klaus, S. J. Ferro, and S. M. George, Thin Solid Films **360**, 145 (2000).
[21] W. Cho, K. Sung, K.-S. An, S. Sook Lee, T.-M. Chung, and Y. Kim, J. Vac. Sci. Technol. A **21**, 1366 (2003).

[22]Y. Liu, D. Li, K. Wang, Z. Xiong, F. Shi, J. Wang, B. Li, and B. Hang, Inf. Process. Manage. **61**, 103809 (2024).
[23]F. Cappello, S. Madireddy, R. Underwood, N. Getty, N. L.-P. Chia, N. Ramachandra, J. Nguyen, M. Keceli, T. Mallick, Z. Li, M. Ngom, C. Zhang, A. Yanguas-Gil, E. Antoniuk, B. Kailkhura, M. Tian, Y. Du, Y.-S. Ting, A. Wells, B. Nicolae, A. Maurya, M. M. Rafique, E. Huerta, B. Li, I. Foster, and R. Stevens, "EAIRA: Establishing a methodology for evaluating AI models as scientific research assistants," arXiv:2502.20309 [cs.AI] (2025).
[24]Y. Chiang, E. Hsieh, C.-H. Chou, and J. Riebesell, "LLaMP: Large language model made powerful for high-fidelity materials knowledge retrieval and distillation," arXiv:2401.17244 [cs.CL] (2024).
[25]A. M. Bran, S. Cox, O. Schilter, C. Baldassari, A. D. White, and P. Schwaller, Nat. Mach. Intell. **6**, 525 (2024).